

Software Project Scheduling



Project Scheduling

- One of the most difficult jobs for a project manager.
- Begins with process decomposition. It involves separating total work involved in a project into separate activities and judging the time required to complete these activities.
- Managers estimate time and resources required to complete activities and organize them into a coherent sequence.

Project Scheduling

- When estimating schedules, you should **NOT** assume that every stage of the project will be problem-free.
 - People may fall ill or may leave
 - Hardware may break down
 - Essential software/hardware may be delivered late
- If the project is new & technically advanced, Then certain parts of it may turn out to be more difficult and take longer than originally anticipated.

Project Scheduling: Why is it important?

- Today's software systems are large, sophisticated and complex
 - Many software engineering tasks occur in parallel
 - Result of work performed during one task may have a profound effect on work conducted in another task.
 - Task interdependencies are very difficult to understand without a schedule
 - It is virtually impossible to assess progress on a large software project without a detailed schedule

Why Are Software Projects Late?

- *There are many reasons why software is delivered late...*
 - An unrealistic deadline established by someone outside the software development group
 - Changing customer requirements that are not reflected in schedule changes
 - Risks not considered at the beginning of the project
 - Miscommunications among project staff resulting in re-work & delay
 - Honest under-estimation of the job

Why Are Software Projects Late?

- Technical difficulties that could not have been foreseen in advance
- Human difficulties that could not have been foreseen in advance
- Project Management fails to track progress –
 - Unaware the project is in behind the schedule & in trouble
 - No corrective actions taken

What to do when face an Unrealistic Deadline?

- Perform a detailed estimate of effort and time using historical data
- Using an incremental process model, develop a strategy to deliver critical functionality by the deadline – document the plan
- Meet with the customer and explain why the project is late
- Offer the incremental development strategy as an alternative

Software Project Scheduling: The Basic Principles

- 1) **Compartmentalization**—Compartmentalize the project into a number of manageable activities, actions & tasks
- 2) **Interdependency**—Indicate task interrelationship
- 3) **Time allocation**- Each task must be allocated some no. of work units with start date & completion date
- 4) **Effort validation**—be sure resources are available
- 5) **Defined responsibilities**—Every task should be assigned to a specific team member
- 6) **Defined outcomes**—Each task must have an output (e.g. work product)
- 7) **Defined milestones**—Task(s) should be associated with a project milestone (A milestone is **accomplished** when one or more work products has be reviewed for quality and approved)



The relationship between People and Effort

- “If we fall behind schedule, we can always add more programmers and catch up later in the project!”

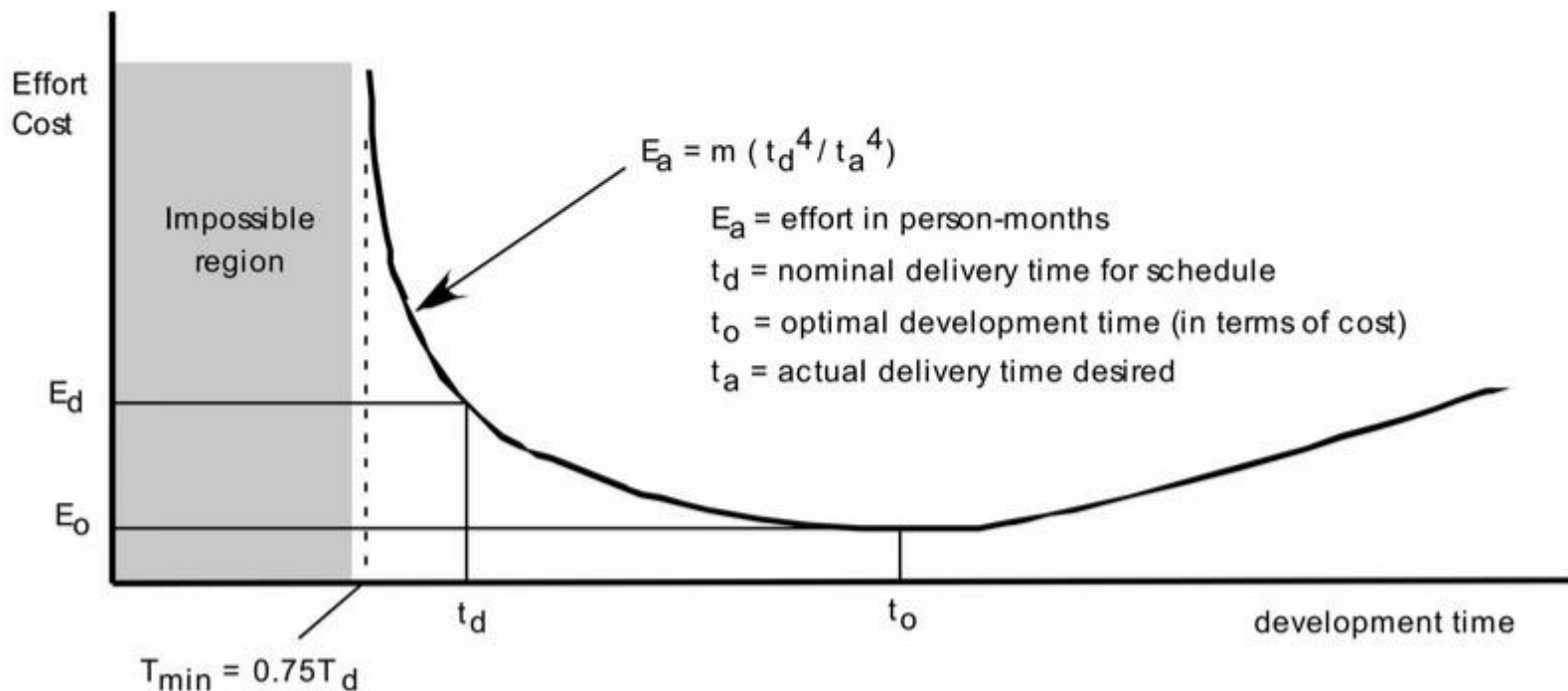
Why does this not work?

- Added/new people must learn the system & learning takes time
- Teaching takes time away from productive work
- # of communication paths & Complexity of communication increase

The relationship between People and Effort

- Over the years, empirical data & theoretical analysis have demonstrated that project schedules are elastic
 - It is possible to compress a desired project completion date by adding additional resources to some extent
 - It is also possible to extend a completion date by reducing the # of resources
- The **Putnam-Norden-Rayleigh (PNR) Curve** provides an indication of the relationship between effort applied and delivery time for a software project.

Relation between effort & delivery time: The PNR Curve



Inferences from the PNR Curve:

- **Impossible region** (left of $T_{min} = 0.75 T_d$) — schedule can't be compressed below ~75% of nominal time, regardless of resources added.
- **Compressing time = exponential cost** — effort rises steeply ($\propto 4$ th power) as you push delivery time toward T_d .
- **t_o = cost-optimal delivery time** — the minimum point of the curve; least effort for the schedule chosen.
- **Delaying past t_o also raises cost, but gently** — far cheaper than compressing.
- **Practical rule:** never accept a deadline inside the impossible region; extending schedule is cheaper than compressing it.



Project Effort Distribution

- *How should effort be distributed across the software process workflow?*
- A recommended distribution of effort across the software process is often referred to as *the 40-20-40 rule*
 - ❑ 40% of effort ==> “Front-end” (analysis & design)
 - ❑ 20% of effort ==> Coding
 - ❑ 40% of effort ==> “Back-end” (testing)

Project Effort Distribution

- *Generally accepted guidelines :*

02-03 % ==> Project Planning

10-25 % ==> Requirements analysis

20-25 % ==> Design

15-20 % ==> Coding

30-40 % ==> Testing and debugging

Define a Task Network

- A Task Network (AKA Activity Network) is a graphic representation of the task flow for a project.
- A task network depicts each software engineering task, its dependency on other tasks, and its projected duration.
 - The task network is used to compute the critical path, a timeline chart and a variety of project information
 - The task network is a useful mechanism for depicting intertask dependencies & determining the critical path.

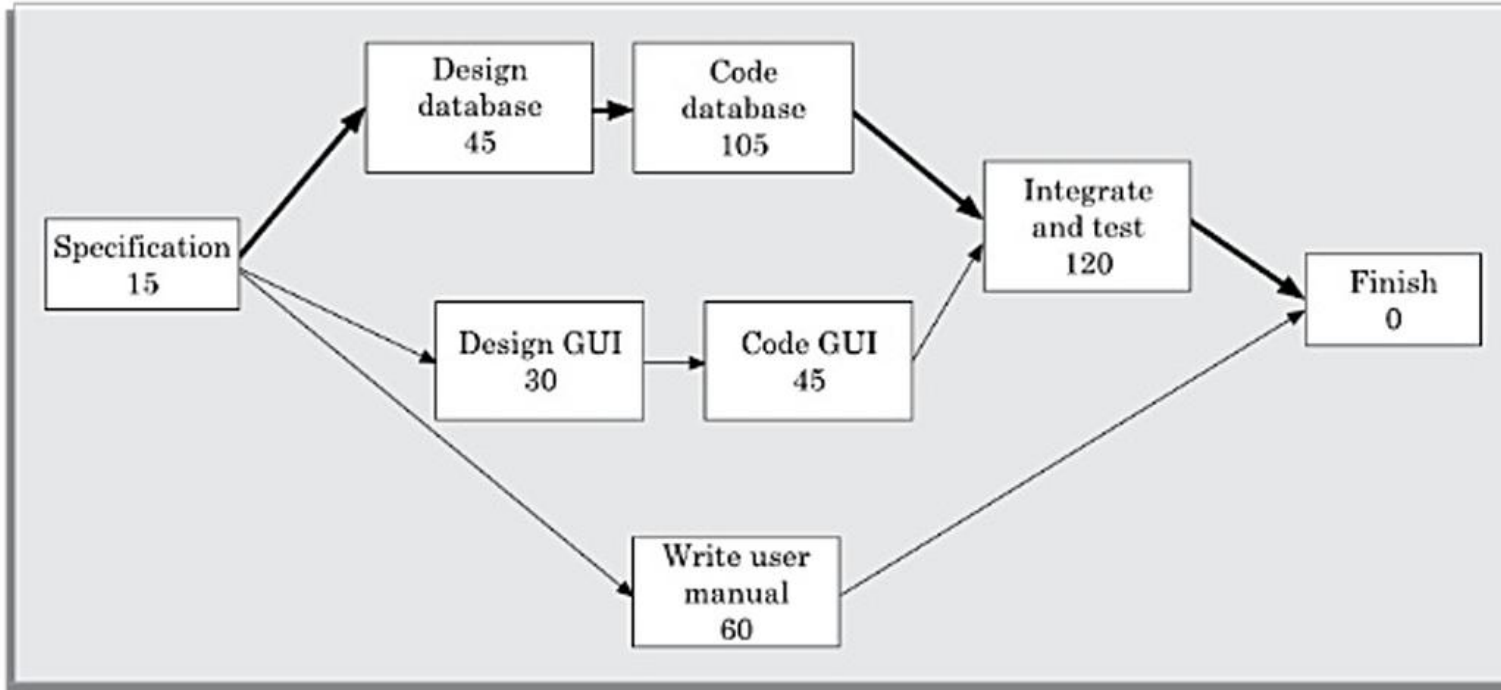


Table 3.7: Project Parameters Computed from Activity Network

Task Number	Task	Duration	Dependent on Tasks
T1	Specification	15	–
T2	Design database	45	T 1
T3	Design GUI	30	T 1
T4	Code database	105	T 2
T5	Code GUI part	45	T 3
T6	Integrate and test	120	T 4 and T 5
T7	Write user manual	60	T 1

- An activity network shows the different activities making up a project, their estimated durations, and their interdependencies.
- Activity on Node (AoN): In this representation, each activity is represented by a rectangular (some use circular) node and the duration of the activity is shown alongside each task in the node.
- The inter-task dependencies are shown using directional edges.

Scheduling Methods

- Two project scheduling methods can be applied to software development-
 - 1) PERT-Program Evaluation and Review Technique
 - 2) CPM- Critical Path Method
- Both techniques are driven by information already developed in earlier project planning activities –
 - Estimates of effort
 - A decomposition of product function
 - Selection of appropriate process and task set
 - Decomposition of task

Scheduling Methods

- Both PERT & CPM provide quantitative tools that allows software planner to –
 - Determine critical path –the chain of tasks that determines project-duration
 - Establish “most likely” time schedules for individual tasks (using statistical model)
 - Calculate “boundary times”

Critical Path Method (CPM)

- Critical Path Method (CPM) is a method used in project planning, generally for project scheduling for the on-time completion of the project.
- It helps in the determination of the earliest time by which the whole project can be completed.
- A critical path consists of a set of dependent tasks that need to be performed in a sequence and which together take the longest time to complete.
- It is a **sequence of critical tasks/activities** and is the largest path in the project network.
- It gives us the **minimum time which is required to complete the entire project**.
- The activities in the critical path are known as critical activities and if these activities are delayed then the completion of the entire project is also delayed.

Benefits of using the critical path method in project management:

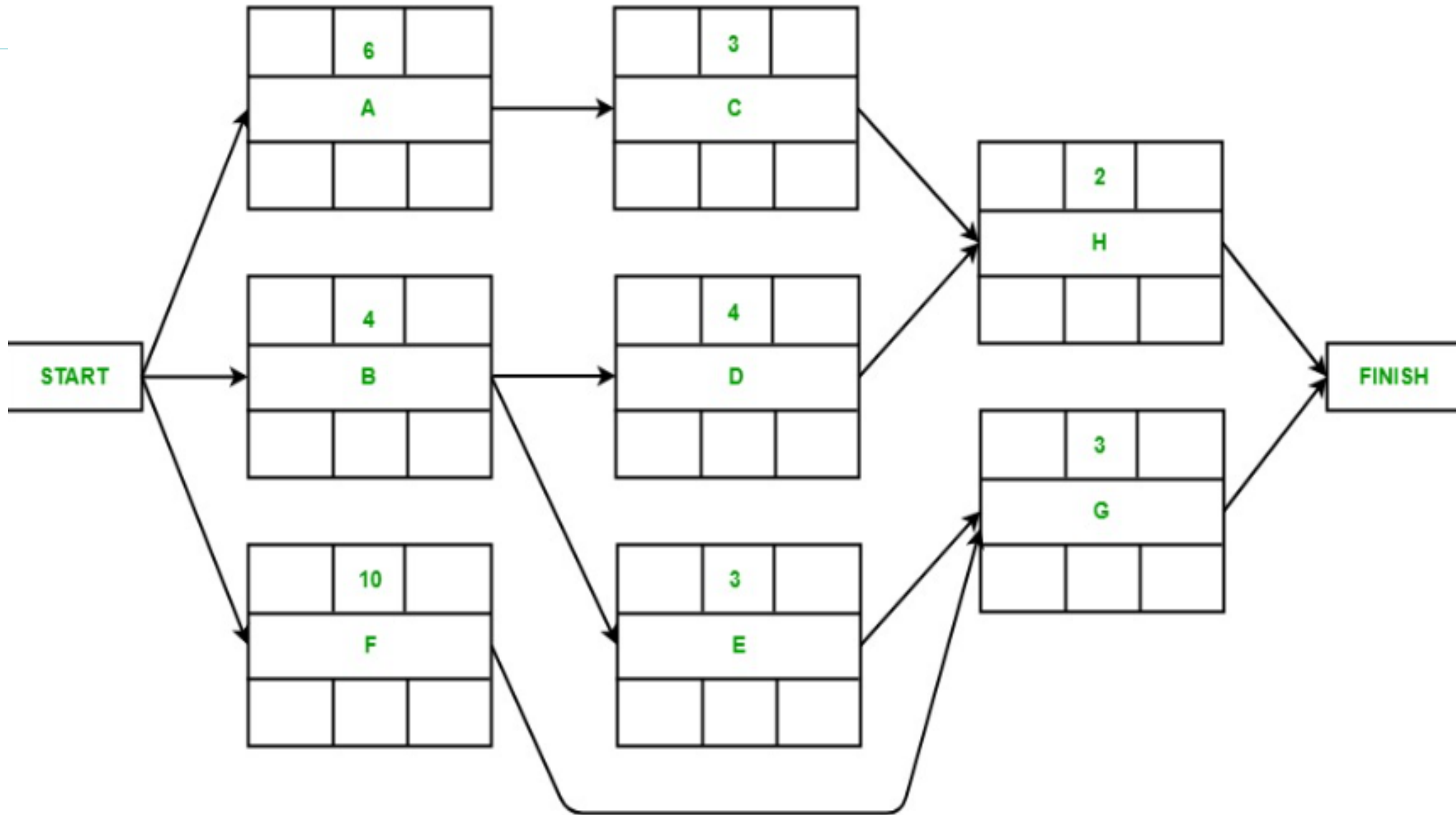
- Show the project schedule visually.
- Highlight important tasks with CPM.
- Use CPM to find and handle risks.
- CPM helps the project team communicate better.

Activity	Duration (in weeks)	Precedents
A	6	—
B	4	—
C	3	A
D	4	B
E	3	B
F	10	—
G	3	E,F
H	2	C,D

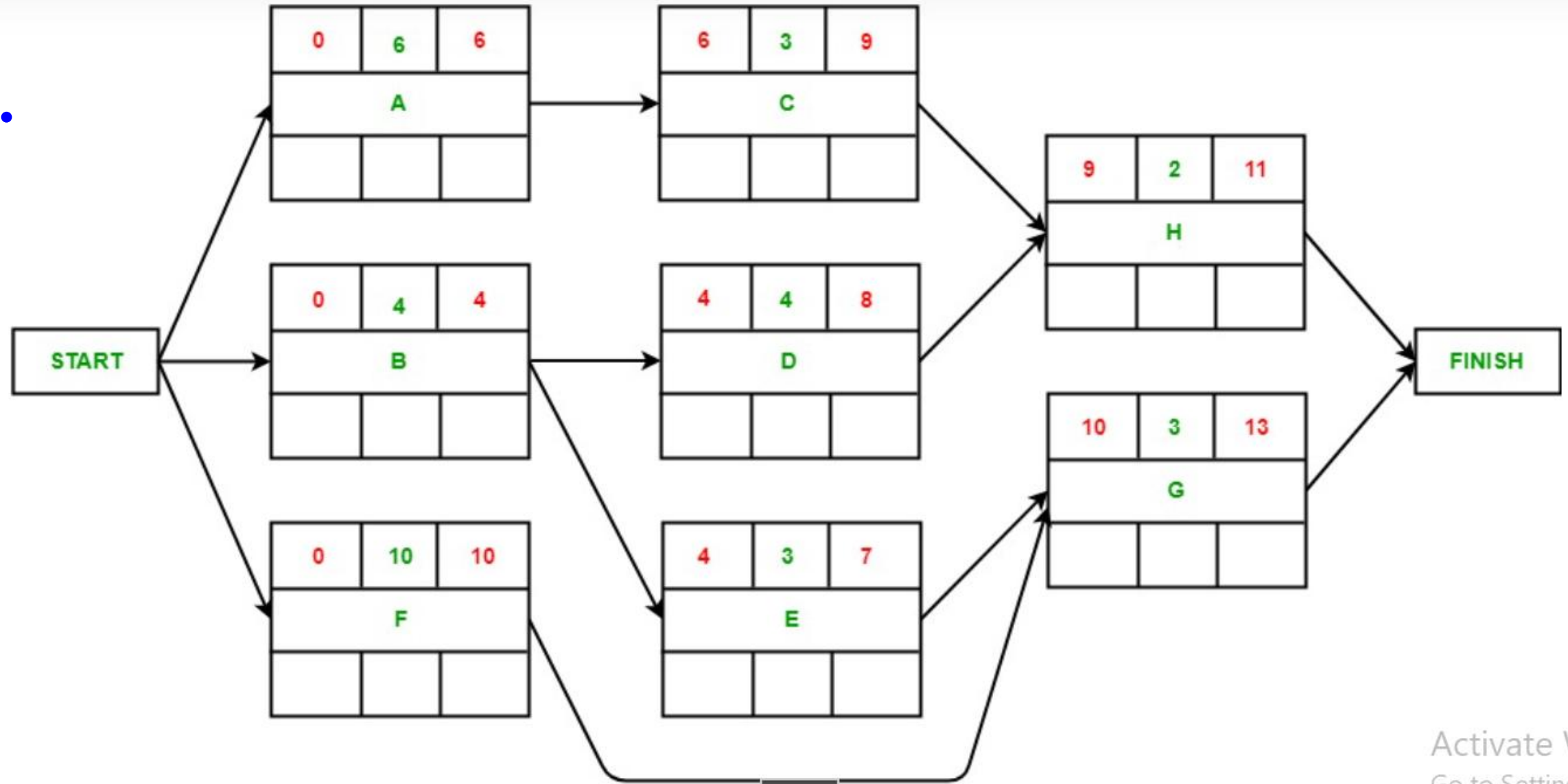
Earliest Start	Duration	Earliest Finish
Activity Label		
Latest Start	Float	Latest Finish

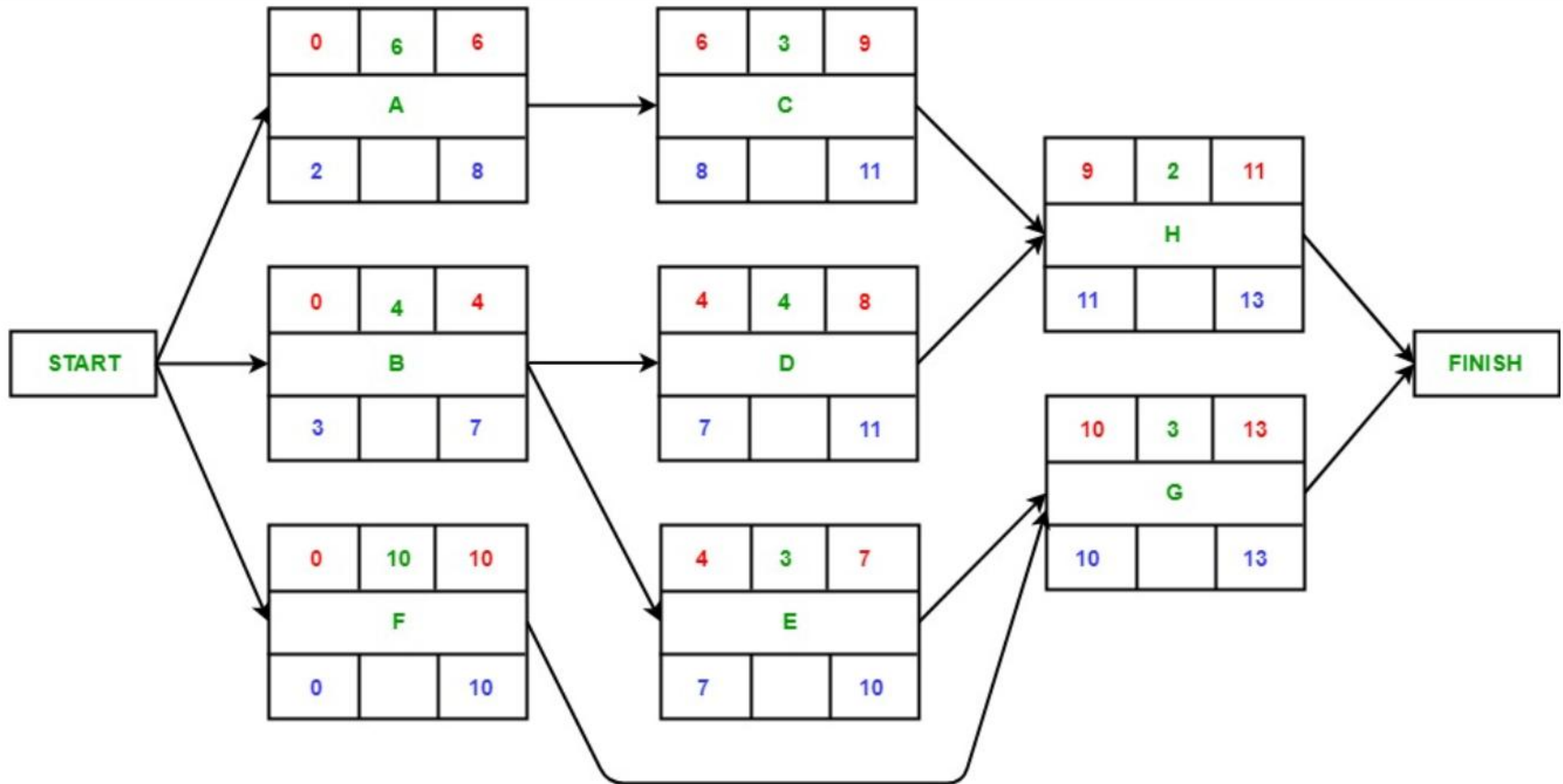
- **Activity label** is the name of the activity represented by that node.
- **Earliest Start** is the date or time at which the activity can be started at the earliest.
- **Earliest Finish** is the date or time at which the activity can be completed at the earliest.
- **Latest Start** is the date or time at which the activity can be started at the latest.
- **The latest Finish** is the date or time at which the activity can be finished at the latest.
- **Float** is equal to the difference between the earliest start and latest start or earliest finish and latest finish.

Activity-On-Arrow diagram:



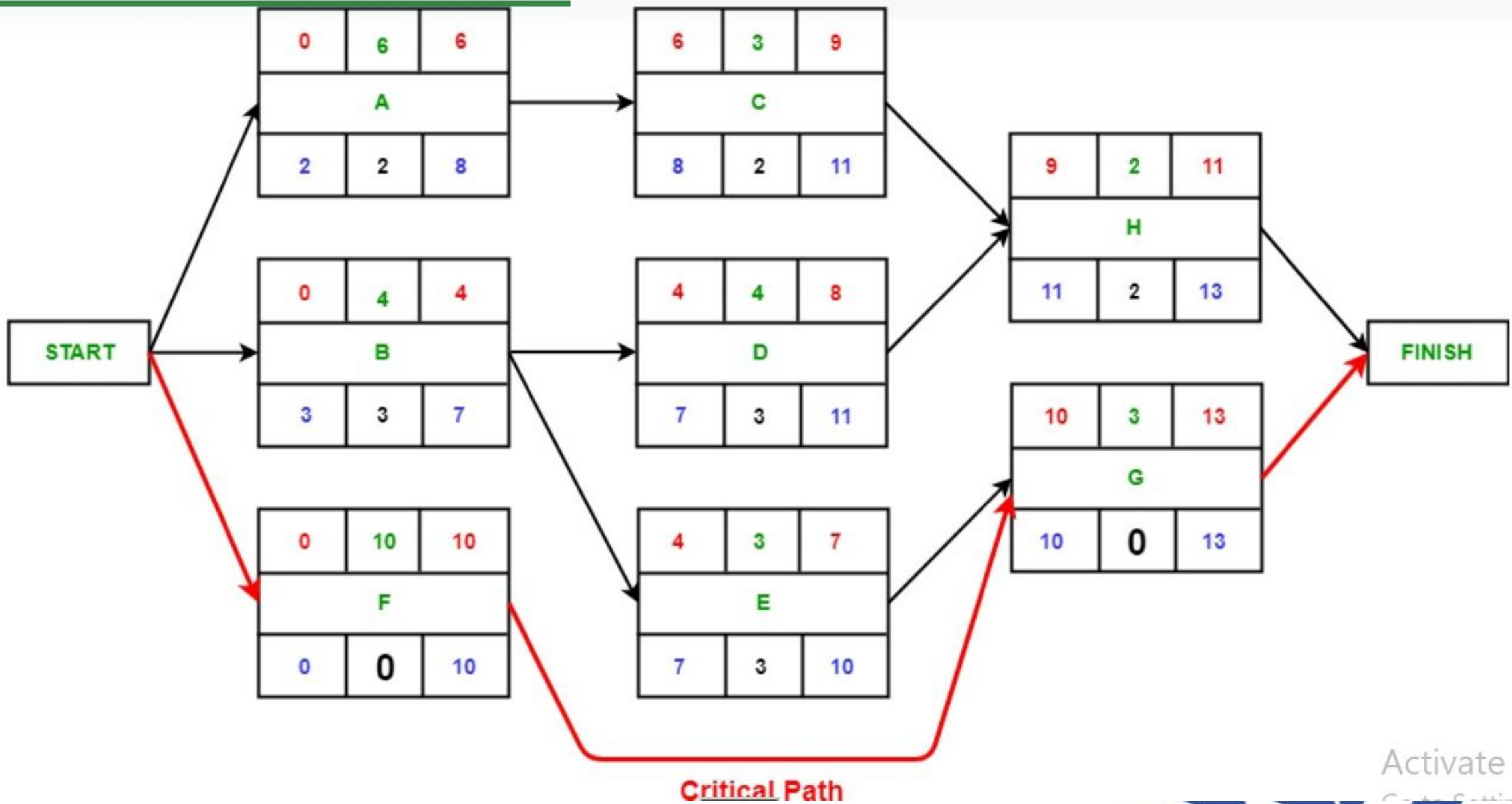
Forward Pass in Critical path in project management:





Backward Pass in Critical path in project management

-
- **Identifying Critical Path:**
 - Activity float is the difference between an activity's Earliest start and its latest start date or the difference between the activity's Earliest finish and its latest finish date.
 - it indicates how much the activity can be delayed without delaying the completion of the entire project.
 - If the **float of an activity is zero, then the activity is critical** and must be added to the critical path of the project network. In this example, activities F and G have zero float and hence, are critical activities.



Project Scheduling Techniques

2. Program Evaluation and Review Technique (PERT):

- It is a way to schedule flow of tasks in a project and estimate total time taken to complete it.
- PERT charts offer a visual representation of the major activities (and dependencies) in a project

It calculate:

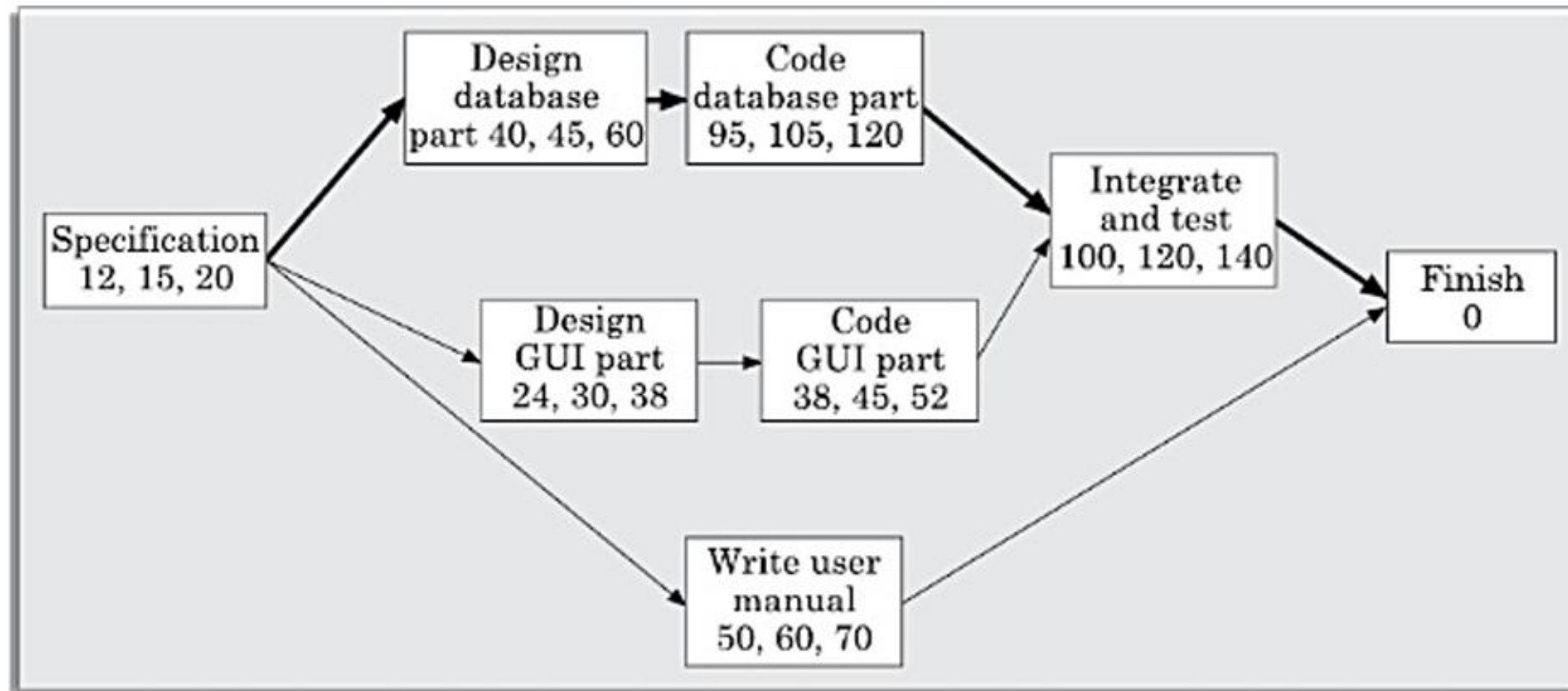
Each task is annotated with three estimates:

Optimistic (O): The best possible case task completion time.

Most likely estimate (M): Most likely task completion time.

Worst case (W): The worst possible case task completion time.

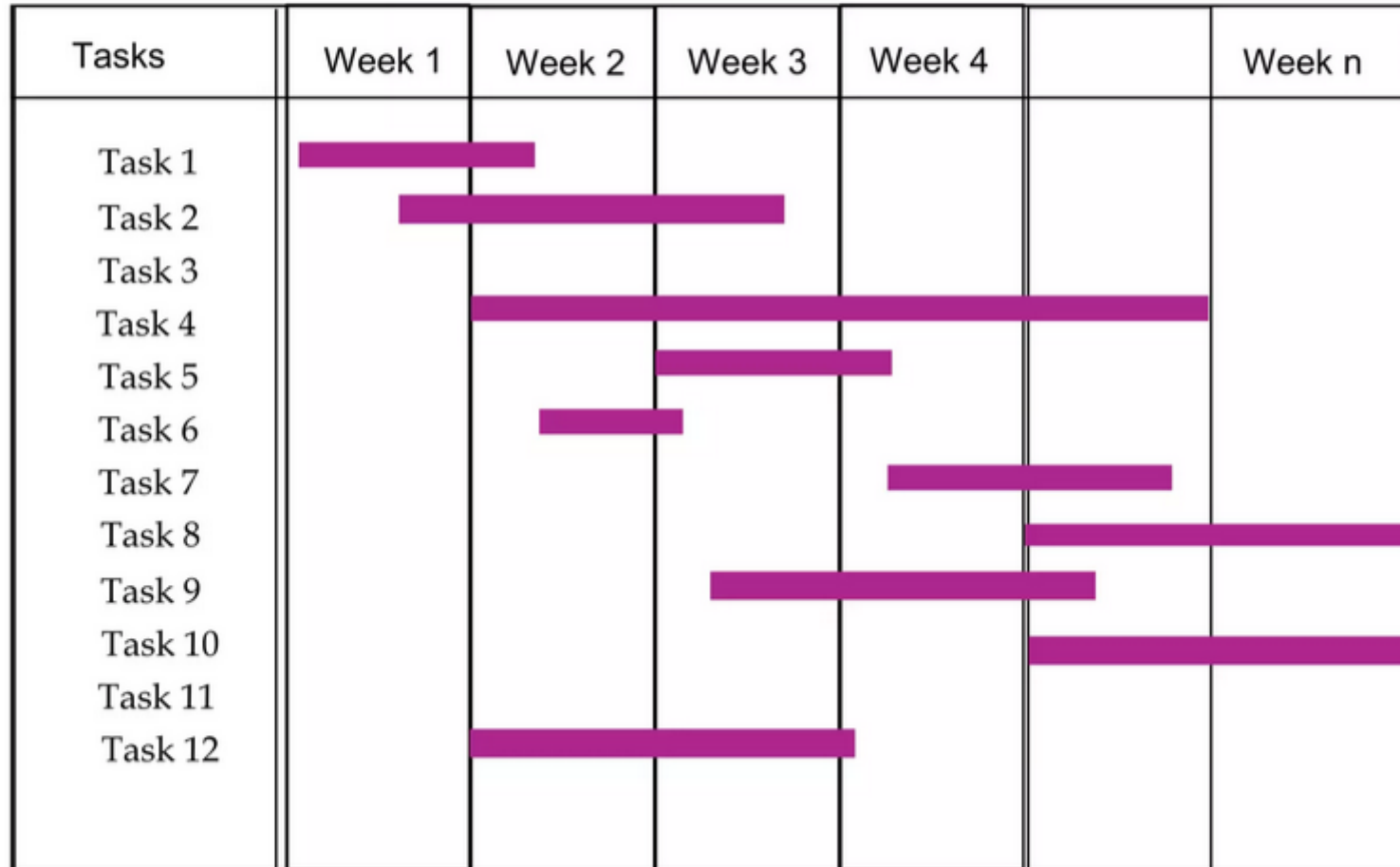
Example



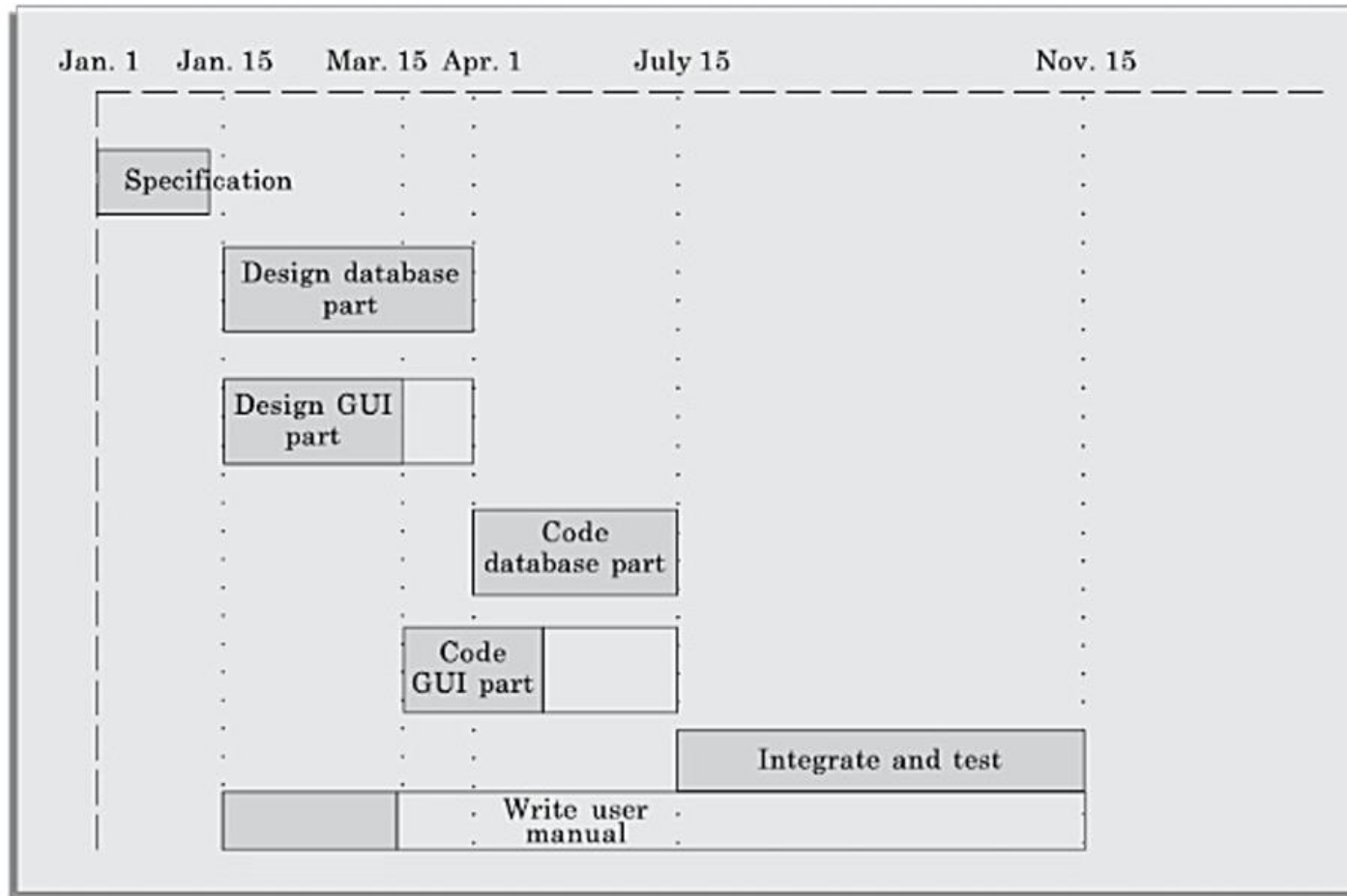
Timeline Charts

- When creating a s/w project schedule, the planner begins with a set of tasks (The work breakdown structure). If automated tools are used, work breakdown is input as a *task network*. As a consequence, a timeline chart is generated.
- A **timeline chart** (also called **Gantt chart**) can be developed for the entire project. Also, separate charts can be developed for each project function.
- A timeline chart enables you to determine what works will be conducted at a given point in time.

Timeline Charts



Timeline Charts



Tracking the schedule

- Project schedule provides a road map for a software project manager. Project schedule defines the tasks & milestones that must be tracked & controlled as the project proceeds.
- ***Schedule Tracking techniques used by Experienced Project Managers:***
 - Conducting periodic project status meeting in which each team member reports progress & problems
 - Evaluating the results of all reviews
 - Determining whether formal milestones have been accomplished by the scheduled date
 - Compare actual start-date with scheduled start-date
 - Informal meeting with practitioners
 - Using earned value analysis to assess progress quantitatively.

Tracking the schedule

- Control is employed by a project manager to administer project resources, cope with problems, and direct project staff
- If things are going on well, control is light.
- If problems occur, project manager must exercise control to reconcile them as quickly as possible.
 - Diagnose the problem
 - Additional resources may be focused on the problem area
 - Staff may be redeployed
 - Project schedule can be redefined

Tracking the schedule

- What is the indication of progress?
 - The best indication of progress is the completion & successful review of a defined software work product.